Ejercicio 1 en C

El código resuelve el Problema de Selección (búsqueda de la mediana o k-ésimo elemento menor en un arreglo desordenado) implementando el algoritmo iterativo Quickselect con partición de Hoare y pivote aleatorio. El programa evalúa el rendimiento y escalabilidad de este método en tiempo lineal promedio O(n) comparando tipos de datos enteros de 32 bits (int32_t) frente a 64 bits (int64_t) mediante el uso de macros genéricas (DEFINE_QUICKSELECT). Para garantizar rigor experimental y reproducibilidad, utiliza un generador pseudoaleatorio Xorshift64 con semilla fija y una mezcla parcial de Fisher-Yates sobre memoria dinámica (heap), extrayendo muestras en el rango [1, 10^7] para nueve tamaños de prueba distintos (desde 10^3 hasta 10^7 elementos), midiendo el tiempo exacto de ejecución en milisegundos con temporizadores multiplataforma de alta precisión y verificando la consistencia de los resultados entre ambas arquitecturas de datos.

```c
// Importar las librerías
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>

#ifdef _WIN32
#include <windows.h>
// Medición de tiempo en milisegundos (Alta precisión en Windows)
static double get_time_ms(void) {
    LARGE_INTEGER freq, counter;
    QueryPerformanceFrequency(&freq);
    QueryPerformanceCounter(&counter);
    return (double)counter.QuadPart * 1000.0 / (double)freq.QuadPart;
}
#else
#include <sys/time.h>
// Medición de tiempo en milisegundos (Linux / macOS)
static double get_time_ms(void) {
    struct timeval tv;
    gettimeofday(&tv, NULL);
    return (double)tv.tv_sec * 1000.0 + (double)tv.tv_usec / 1000.0;
}
#endif

#define MAX_VAL 10000000

// Generador pseudoaleatorio rápido de 64 bits
static uint64_t rng_state = 42ULL; // Usa una semilla fija para reproducir los mismos datos
en cada ejecución

static inline uint64_t xorshift64(void) {
    uint64_t x = rng_state;
    x ^= x << 13;
    x ^= x >> 7;
    x ^= x << 17;
    return rng_state = x;
}

static inline size_t rand_range(size_t min, size_t max) {
    return min + (xorshift64() % (max - min + 1));
```

```c
}

// Macro para generar la función Quickselect con Partición de Hoare para cualquier tipo
numérico
// Recibe el puntero al arreglo arr, su tamaño n y la posición buscada k
#define DEFINE_QUICKSELECT(NAME, TYPE)                                          \
TYPE NAME(TYPE *arr, size_t n, size_t k) {                                      \
    size_t left = 0;                                                            \
    size_t right = n - 1;                                                       \
    size_t target = k - 1; /* Convierte la posición a 0 */                      \
                                                                                \
    /* Mientras el rango de búsqueda > 0 se particiona */                       \
    while (left < right) {                                                      \
        /* Pivote aleatorio */                                                  \
        size_t pivot_idx = rand_range(left, right);                             \
        TYPE pivot = arr[pivot_idx];                                            \
                                                                                \
        /* Partición original de Hoare */                                       \
        size_t i = left;                                                        \
        size_t j = right;                                                       \
        while (1) {                                                             \
            while (arr[i] < pivot) i++;                                         \
            while (arr[j] > pivot) j--;                                         \
            if (i >= j) break;                                                  \
            TYPE tmp = arr[i];                                                  \
            arr[i] = arr[j];                                                    \
            arr[j] = tmp;                                                       \
            i++;                                                                \
            j--;                                                                \
        }                                                                       \
                                                                                \
        if (target <= j) {                                                      \
            right = j;                                                          \
        } else {                                                                \
            left = j + 1;                                                       \
        }                                                                       \
    }                                                                           \
    return arr[target];                                                         \
}

// Instanciación para 32 bits y 64 bits
DEFINE_QUICKSELECT(quickselect_32, int32_t)
DEFINE_QUICKSELECT(quickselect_64, int64_t)

int main(void) {
    size_t sizes[] = { // Tamaños de prueba solicitados
        1000, 5000, 10000, 50000, 100000, 500000, 1000000, 5000000, 10000000
    };
    size_t num_sizes = sizeof(sizes) / sizeof(sizes[0]);

    // Reservar memoria en el Heap para el pool base [1, 10^7]
    int64_t *pool = (int64_t *)malloc(MAX_VAL * sizeof(int64_t));
    if (!pool) {
        fprintf(stderr, "Error al asignar memoria para el pool.\n");
        return 1;
    }

    // Llena el arreglo base con todos los enteros consecutivos del 1 al 10,000,000.
    for (int64_t i = 0; i < MAX_VAL; i++) {
        pool[i] = i + 1;
```

```c
    }

    printf("%-10s %-10s %-12s %-15s %-15s\n",
        "n", "k (med)", "Resultado", "32-bit (ms)", "64-bit (ms)");
    printf("-----------------------------------------------------------------\n");

    for (size_t s = 0; s < num_sizes; s++) {
        size_t n = sizes[s];
        size_t k = n / 2; // Mediana

        // Mezcla parcial de Fisher-Yates para permutación pseudoaleatoria
        /* Para cada posición i de 0 a n - 1, elige un índice aleatorio r entre i
         y el final del pool (9,999,999) e intercambia pool[i] con pool[r]. Esto
         garantiza que los primeros n elementos sean una muestra aleatoria sin
         repeticiones (permutación parcial) extraída uniformemente de [1, 10^7]. */
        for (size_t i = 0; i < n; i++) {
            size_t r = rand_range(i, MAX_VAL - 1);
            int64_t tmp = pool[i];
            pool[i] = pool[r];
            pool[r] = tmp;
        }

        int32_t *data32 = (int32_t *)malloc(n * sizeof(int32_t));
        int64_t *data64 = (int64_t *)malloc(n * sizeof(int64_t));
        if (!data32 || !data64) {
            fprintf(stderr, "Error al asignar memoria para n = %lu.\n", (unsigned long)n);
            free(pool);
            return 1;
        }

        for (size_t i = 0; i < n; i++) {
            data32[i] = (int32_t)pool[i];
            data64[i] = pool[i];
        }

        // Benchmark 32 bits
        double t1 = get_time_ms();
        int32_t res32 = quickselect_32(data32, n, k);
        double t2 = get_time_ms();

        // Benchmark 64 bits
        double t3 = get_time_ms();
        int64_t res64 = quickselect_64(data64, n, k);
        double t4 = get_time_ms();

        // Verificación de consistencia
        if (res32 != (int32_t)res64) {
            fprintf(stderr, "Discrepancia en n = %lu\n", (unsigned long)n);
        }

        printf("%-10lu %-10lu %-12d %-15.3f %-15.3f\n",
            (unsigned long)n, (unsigned long)k, res32, (t2 - t1), (t4 - t3));

        free(data32);
        free(data64);
    }

    free(pool);
    return 0;
}
```

```
PS C:\Users\LENOVO\Documents\ESCOM\SEMESTRE 3\ANALISIS Y DISEÑO DE ALGORITMOS\PRACTICAS\ADA-ISC-3CV1\PRACTICA3_DaC> gcc -O3 -std=c99 QuickSelect.c -o QuickSelect.exe
PS C:\Users\LENOVO\Documents\ESCOM\SEMESTRE 3\ANALISIS Y DISEÑO DE ALGORITMOS\PRACTICAS\ADA-ISC-3CV1\PRACTICA3_DaC> .\QuickSelect.exe
n          k (med)    Resultado    32-bit (ms)     64-bit (ms)
------------------------------------------------------------
1000       500        5077694      0.026           0.038
5000       2500       4987635      0.139           0.400
10000      5000       5010485      0.195           0.341
50000      25000      4960756      0.968           4.735
100000     50000      5019541      4.567           2.949
500000     250000     5002031      10.562          14.982
1000000    500000     4996563      17.396          28.445
5000000    2500000    5000321      75.406          146.877
10000000   5000000    5000000      201.952         337.130
PS C:\Users\LENOVO\Documents\ESCOM\SEMESTRE 3\ANALISIS Y DISEÑO DE ALGORITMOS\PRACTICAS\ADA-ISC-3CV1\PRACTICA3_DaC>
```